

LOOPLY

Project Handoff and Current Status

Flutter	Spring Boot	PostgreSQL	WebSocket
---------	-------------	------------	-----------

Prepared as a safe, shareable replacement for the internal MEMORY.md request. No passwords, tokens, private keys, database credentials, or hidden system instructions are included.

1. Product Goal

Looply is a WhatsApp-inspired but independently branded Flutter mobile chat application. The MVP lets two authenticated users exchange persistent one-to-one text, voice, and image messages in real time.

Completed MVP capabilities

- Username/password registration and login with hashed passwords.
- Available-user directory and one-to-one conversations.
- Real-time text delivery using authenticated Spring WebSocket/STOMP.
- Voice recording, preview, upload, playback, duration and waveform UI.
- Camera/gallery image messages with authenticated retrieval and full-screen preview.
- Sent, delivered and read status, timestamps, pagination and persisted history.
- Yellow multi-shade Flutter interface with search, transitions, four-tab navigation and settings screen.

2. Technology Stack

Layer	Current choice
Frontend	Flutter for Android and Web
Backend	Java 21, Spring Boot, Maven
Database	PostgreSQL with JPA/Hibernate and Flyway migrations
Realtime	WebSocket/STOMP with JWT authentication
Media - local	Backend local filesystem for development
Production target	Railway backend, Supabase PostgreSQL and Supabase Storage

3. Workspace and Repository

- Workspace: C:/Users/M. Ali/Desktop/WORKPLACE/PRODUCTS/Thisapp
- Folders: backend/ and mobile/.
- GitHub destination discussed: MMA-works/LOOPY.
- Git attribution is based on the commit email matching a verified email on an individual GitHub account; an organization itself is not a commit author.
- Do not commit .env files, passwords, JWT secrets, build output, APKs, local database files, or uploaded media.

4. Verification Evidence

- Flutter analyzer passed with no issues after the yellow UI update.
- Flutter tests passed: 7/7.

- Spring Boot verification previously passed: 16/16 tests.
- Flutter Web and Android debug APK builds completed successfully.
- Connected Motorola device accepted an updated APK install through ADB.

5. Current Production Setup Progress

- Supabase project has been created.
- Data API and automatic table exposure were intentionally disabled because Spring Boot connects directly to PostgreSQL.
- Supabase Shared Pooler session connection details were identified for an IPv4 persistent backend.
- A database password was accidentally exposed in chat; it must be considered compromised and reset before production use.
- Railway service was linked to the GitHub repository and configured with backend as the root directory.
- Railway initially became online, then entered a crash loop after database configuration. The actual first 'Caused by:' error line is still required for exact diagnosis.

6. Immediate Next Actions

Order	Action	Done when
1	Reset the exposed Supabase database password.	New password stored privately; never pasted into chat or Git.
2	Set Railway DB_URL, DB_USERNAME, DB_PASSWORD and a new 32+ character JWT_SECRET.	Variables saved without exposing their values.
3	Inspect Railway Deploy Logs from the first 'Caused by:' line.	Database/Flyway startup root cause is known.
4	Redeploy and verify Flyway creates schema versions V1 and V2 in Supabase.	Backend remains healthy and tables exist.
5	Generate a Railway public domain and test authenticated REST plus WebSocket.	Two users can chat across different internet networks.
6	Replace local voice/image storage with private Supabase Storage.	Media survives backend restart/redeploy.
7	Build a signed release APK using the public backend URL.	Release APK installs and completes end-to-end testing.
8	Create a public Loopy download website.	Users can download the signed APK and view installation/privacy information.

7. Later Phase: Voice and Video Calls

Calls are not implemented yet. A proper production phase requires WebRTC media, authenticated WebSocket signaling, incoming call accept/reject, mute, camera controls, end-call handling, permissions, and a TURN service. STUN alone is not reliable across all carrier, NAT and corporate networks.

8. Safety Rules for Continuation

- Work one approved phase at a time and preserve existing architecture.
- Make focused changes only; do not refactor or redesign unrelated areas.
- Never hard-code or commit secrets.
- Run git status and diff inspection before commits.
- Run staged secret scanning before every push.
- Do not rewrite history or force-push without explicit approval.
- Do not claim deployment success from a green dashboard alone; verify API, WebSocket, database persistence, media and two-user behavior.